自然言語から物理動作へ　シミュレータから実機へ移動するAIPLモバイルエージェントによるロボット操作

児玉 靖司
法政大学経営学部

論文草稿　2026年10月11日（モバイルエージェント方式へ改稿）

要旨

自然言語による指示はロボットのプログラミングを容易にする可能性がある。しかし、生成プログラムが特定の実機で実行可能とは限らない。先行研究には、LLMによるロボットコード生成、検査可能なプログラム、シミュレーション評価がすでに存在する。本稿は、LLMが生成するAIPLの作業プログラムを*モバイルエージェント*として扱う構成を提案する。エージェントはまずシミュレータを主（ホスト）として送られて実行され、検査とシミュレーションに合格した場合に限り、*同じソース*と自らの記憶を携えて実機の主へ移動し、そこで実行される。各主はエージェントに「いま居る主の腕」を動かす同名の基本操作だけを与え、機器固有の制御はエージェントの外側に置く。シミュレーションによる事前選別は、この移動を許すかどうかの関門として定式化される。2026年10月11日、立方体を手首カメラで探して挟み、色で左右に置く作業を一つのエージェント（174行）に書き、ブラウザ上のXinuシミュレータへ送って緑と青の2個を運ばせた後、同じソースを実機（bare-metalのEmbedded Xinuが動くRaspberry Pi 5上の6軸アーム）へ移動させ、実機でも2個とも運んだ。両方の主で35段を要し、所要時間はシミュレータで107秒、実機で144秒であった。実機のXinuは一回の実行を250 msで打ち切るため、エージェントを短い断片を一つずつ進める状態機械として書き、待機とカメラへの依頼はMac上の中継（港）が断片の間に行う。同じソースを二つの主で動かすために、処理系の差6件と運用上の問題3件を揃える必要があった。以前の方式（プログラムをMacに置いたまま、腕の宛先だけを替える遠隔呼出し方式）で得た結果も併せて報告する。未見の指示30件による生成の予備実験では27件が型検査を通り、接触を解く検証段は運動学だけの段が見逃す4件を検出した。実機を動かす過程で、運動学の模型が見逃す差を八つ同定し、「見えない」を「挟めた」とする判定の誤りを指の読み戻しに置き換えた。手順を固定した反復では、1個の作業の完遂率は0.75、2個では0.3であった。シミュレーションによる事前選別の効果を実機で測る主実験は未実施である。

キーワード：ロボット操作、言語モデル、プログラム生成、Actorモデル、モバイルエージェント、AIPL、シミュレーション、Sim-to-Real

1　はじめに

人は関節角度の列ではなく、物体をどこへどの順番で置きたいかを自然言語で伝える。この指示をロボットの動作へ変えるには、指示中の物体の同定、操作順序、到達可能な把持姿勢と経路の選択、周囲との干渉の確認が必要である。既存研究は言語モデルがロボット向けのプログラムを組み立てられることを示したが、流暢なコードは特定の作業空間での実行可能性を保証しない[2, 3]。実行時には認識誤差、較正誤差、接触の不確実性も加わる。シミュレーションは実機を動かす前に一部の問題を発見し得る一方、シミュレーションと実機の一致度は対象の作業ごとに評価しなければならない[5]。

本稿が扱う流れは、自然言語の指示、観測された場面、LLMが生成したAIPLプログラム、シミュレーション、条件付きの実機実行である。AIPLは著者が開発するActor指向言語であり、本稿では作業論理を検査可能な形で表現する用途に注目する。LLMは生のモータ指令を生成せず、作業レベルの操作を呼び出す作業プログラムを生成する。

本稿はこの作業プログラムを*モバイルエージェント*として扱う。エージェントは、まずシミュレータを主として送られ、そこで実行される。シミュレーションに合格したときに限り、同じソースが自らの記憶を携えて実機の主へ移動し、実機の腕を動かす。どちらの主も、エージェントに「いま居る主の腕」を表す同名の基本操作を与える。したがってエージェントのソースには、自分がどちらの主に居るかを示す記述が無

い。シミュレーションによる事前選別は、エージェントの*移動を許すかどうかの関門*になる。以前の版の本稿は、プログラムをMacに置いたまま腕の宛先だけを替える遠隔呼出し方式を採っていた。本版はモバイルエージェント方式へ改め、以前の方式で得た結果（7.1–7.6節）はその旨を明記して残す。

本稿の貢献は検証可能なシステム設計と評価手順であり、シミュレーションによる安全性の保証を主張するものではない。研究課題は次の四つである。（RQ1）LLMは指示に沿う、構文上・型上正しいAIPLプログラムを生成できるか。（RQ2）*同じAIPLソースのエージェント*が、シミュレータの主から実機の主へ移動して両方で実行できるか。また観測可能な実行履歴はどこで異なるか。（RQ3）静的検査とシミュレーションは、それぞれどの種類の不正・実行不能なプログラムを除外できるか。（RQ4）同じ指示と場面の下で、事前選別は全指示の完了率、実機での失敗、投入率、追加時間へどう影響するか。RQ1には生成の予備実験（7.1節）、RQ2にはモバイルエージェントの移動実行（7.7節）と以前の方式での実行（7.3–7.6節）、RQ3には検証段の比較（7.2節）と実機で見つかった模型の見逃しで部分的に答える。RQ4は主実験を要するため未実施である。

2　関連研究

言語からロボットプログラムへ。　SayCanは言語モデルによる計画とロボットの技能実行可能性の推定を組み合わせる[1]。Code as Policiesは自然言語と例からロボット向けの実行可能なプログラムを生成し、実機での作業も示した[2]。VoxPoserは言語による推論、知覚、三次元の価値マップを運動計画に組み合わせ、シミュレーションと実機の両方で評価した[3]。これらを踏まえ、本稿で検証したい特徴は、AIPLのActor表現、シミュレーションと実機で共通の作業インターフェース、および*同一の生成プログラム*に対する事前選別の効果である。これらを性能上の優位性として主張するには実験が必要である。

シミュレーションと移行。　シミュレーションは作業、データ、評価条件の生成に利用されている。GenSim2はコード生成能力を持つLLMを使ってシミュレーションの作業とデータを作成する[4]。SIMPLERはシミュレーションと実機の対応する評価を実施し、検討した条件では両者の性能に有用な相関があると報告した[5]。本稿ではシミュレータを生成AIPLプログラムの実行可否を判断する段階として用い、その予測力を対応する実機試行で測る。CG表示が似ているだけでは、衝突、把持、力学の検証が成立したことにはならない。

プログラム合成とシミュレーションによる修正。　LDIPSは読みやすいドメイン固有言語で行動選択プログラムを合成し、ロボットサッカーと自動運転の分野で、シミュレーションから実機への移行時のプログラム修正を示した[6]。WangらはLLMを用いた静的・テキストベースのシミュレーションでロボット操作コードを修正する方式を研究し、ドローンと地上車両の課題で評価した[7]。したがって、読めるプログラムやシミュレーションからの修正そのものを本稿の新規性とはしない。AIPLのメッセージ契約、同じ作業プログラムの二つのバックエンドでの実行、静的検査・物理シミュレーション・誤った合格判定を切り分けた実験の組合せを検証する。差別化は実装と実験が済むまでは仮説である。

モバイルエージェント。　コードが実行場所を移る方式は、移動するものによって分類されてきた。Fuggettaらは、コードと実行状態の両方を運ぶ強い移動性と、コードと明示的に保存したデータだけを運ぶ弱い移動性を区別した[8]。Aglets[9]などのモバイルエージェント基盤は、エージェントが自ら主を渡り歩く構成を示し、LangeとOshimaはその利点として通信量の削減や非同期の自律実行などを挙げた[10]。AIPLはActorモデル[11]に基づき、アクターはメッセージによってのみ相互作用する。本稿はこの考え方を、シミュレータから実機への移行という一回の移動に用いる。本稿の移動は、ソースと明示的に保存した記憶だけを運ぶ弱い移動性であり、実行途中の状態は運ばない。移動そのものを新規性とは主張しない。主張の候補は、シミュレーションへの合格を移動の条件とすることと、移動先の実機で同じソースが動くことを測ることにある。

比較の範囲。　提案方式はActor APIの背後に再利用可能な把持・動作サービスを置き、関節レベルの制御器や汎用視覚運動方策を合成するものではない。主たる比較はシミュレーション選別を取り除く内部比較である。副次的に、作業レベルの操作APIを揃えてPythonの作業コードを生成するCode as Policies型の比較

手法を用意し、言語モデル、知覚入力、例示数、機体、指示集合を揃える。これは原論文の全機能の再現ではなく、作業表現とコンパイルの影響を調べる比較である。

3　新規性の候補と必要な証拠

移動するエージェントとしての作業プログラム。　生成した作業プログラムを、シミュレータの主で実行し、合格したときだけ同じソースのまま実機の主へ移動させる。移動の前後でソースが一字も変わらないこと、主ごとに異なるのは主が与える基本操作の実装だけであることを示す。同じソースを二つの主で動かすために揃えた処理系の差を、すべて列挙する（7.7節）。

AIPLのActor契約。　観測、動作要求、受理通知、完了通知、種別付きの失敗を定義し、適切なメッセージには物体ID、場面の版、撮影時刻、要求ID、座標系を含める。変更のない同一のAIPLソースが両バックエンドで動くことを示す。対象言語への変換やリンク方法は異なってよい。共通のソース、生成した対象言語コード、バックエンド固有アダプタの行数を報告する。AIPLという名称だけでは設計上の貢献を証明できない。

段階的な検証。　却下理由を、構文・型、Actor利用規則、幾何・物理シミュレーション、独立した実機ゲートに分ける。合格したプログラムについて実行履歴と最終目標を照合し、誤った合格判定の原因を記録する。CGの表示だけを物理的な検証の証拠とはしない。

事前選別の効果。　主要比較では初回に生成したプログラムを固定する。*全指示*を分母とする完了率、実機の失敗、投入率、検証時間、投入した試行だけの成功率をともに測る。失敗試行の減少と作業完了数の増加は同義ではないため、優位性は前提としない。

4　問題設定

自然言語の指示をu、初期場面を$s = (O, G, W)$とする。Oは観測した物体、Gは目標領域、Wはロボットの作業空間である。生成器G_θはActor契約Aの下で候補AIPLプログラム$p = G_\theta(u, s, A)$を生成する。コンパイルと静的検査の結果を$C(p) \in \{0, 1\}$とする。シミュレーションの実行履歴を$\tau_{\mathsf{sim}} = \mathsf{Exec}_{\mathsf{sim}}(p, s)$、その履歴と指示に対する検証結果を$V(\tau_{\mathsf{sim}}, u) \in \{0, 1\}$とすると、実機への投入条件は次式となる。

$$\mathsf{Deploy}(p) = \mathbf{1}\{C(p) = 1 \wedge V(\tau_{\mathsf{sim}}, u) = 1 \wedge H(p, s) = 1\}. \tag{1}$$

Hは作業空間、速度、把持力、操作者の準備状態を確認する独立した投入ゲートである。モバイルエージェント方式では、エージェントを$a = (p, \sigma)$と書く。pはソース、σはエージェントが自ら保存して運ぶ記憶である。主$h \in \{\mathsf{sim}, \mathsf{real}\}$での実行を$\mathsf{Exec}_h(p, \sigma, s)$とし、$\mathsf{Deploy}(p) = 1$のときに限り、$a$をシミュレータの主から実機の主へ移す。移動で運ぶのは一字も変えないソースpと記憶σであり、実行途中の状態は運ばない。実機の実行履歴は$\tau_{\mathsf{real}} = \mathsf{Exec}_{\mathsf{real}}(p, \sigma', s')$とし、$\sigma'$はシミュレータの主での実行を終えた時点の記憶、$s'$は実行直前に再観測した場面とする。$\sigma'$にシミュレーションで得た場面の推定を入れて実機で再利用することはしない。sからs'への変化が定めた許容範囲を超えれば、それまでのシミュレーション結果を無効にし、再検証する。

作業成功関数$Y(u, s, \tau)$は観測可能な最終配置と要求された操作順序から定義する。シミュレーションで合格したのに実機で$Y(u, s', \tau_{\mathsf{real}}) = 0$となる場合、または対応する中間状態のずれが閾値を超える場合をSim-to-Realの不一致とする。この定義によって、候補の却下、実機作業の完了、シミュレーションの誤った合格判定を区別できる。採択された試行だけの成功率を示して、却下した指示の割合を隠してはならない。

5　提案システム

5.1　処理の流れと役割の境界

生成器へ渡す情報は、指示、観測した物体と領域の限定的な記述、許可されたAIPL Actorメソッド、および開発用データだけから選んだ例である。出力はAIPLプログラムと、想定した物体識別子と目標条件の機械可読な一覧である。構文解析器と型検査器は、構文の誤りや許可されていない呼出しを却下する。合格したプログラムはエージェントとしてシミュレータの主へ送られ、そこで実行・検証される。合格すれば同じソースが実機の主へ移動する。機器固有の較正と運動計画は、各主が与える操作の側に置き、生成された作業論理の外側に保つ。

自然言語の指示と観測場面 $\longrightarrow$ LLMが生成するAIPLエージェントp
$\downarrow$
構文・型・Actor利用規則の検査
$\downarrow$ エージェントをシミュレータの主へ送る
シミュレータの主で実行 $\longrightarrow$ 実行履歴と作業条件の検証
$\downarrow$ 合格し、場面が変わっていない場合：同じpと記憶σ'が移動する
実機の主で実行（制約付き） $\longrightarrow$ 観測された実機の結果

LLMは操作順序、条件分岐、利用可能な物体や領域の選択を担当する。サーボのトルク設定や時間制約の厳しいフィードバック制御は担当しない。座標変換、逆運動学、軌道の実行、グリッパーの制御、安全停止はバックエンドが担う。この境界により、機器固有の詳細を生成プログラムから分離し、失敗原因を生成、知覚、経路計画、物理実行に分けて分析できる。

5.2　モバイルエージェントの実行モデル

主が与えるもの。　主は二つある。シミュレータの主は、Mac上のブラウザXinuデスクトップ（aice-avm）のAIPL実行系で、運動学の腕の模型とCGの立方体を持つ。実機の主は、DOFBOTのRaspberry Pi 5上で動くbare-metalのEmbedded Xinuで、機内のAIPLコンパイラが到着したソースを機械語に変えて実行する。エージェントの受け入れ口は、シミュレータでは`POST /actor/loadsrc`、実機では`POST /cc`であり、どちらも*ソースの文字列*を受け取って到着先でコンパイルする。両方の主は、腕への文字列命令を受ける組込み関数`arm_cmd(s)`を同じ名前で与える。シミュレータではこれが模型の腕に、実機ではI^2Cのサーボ基板に結び付く。エージェントはこの関数だけで腕を動かし、宛先を書かない。

断片ごとに進める状態機械。　実機のXinuは、機内コンパイラで動かすメソッドの一回の実行を250 msで打ち切り、その経路での待機（`wait`）を許さない。単一スレッドのHTTPやネットワーク処理を塞がないためである。数分かかる作業をエージェントが一度の実行で進めることはできない。そこでエージェントを状態機械として書き、メソッド`step(input)`の一回の呼出しで、印`// [n]`を付けた断片を一つだけ実行する。断片は腕へ命令を一つ送るか、判断を一つ下して、次の要求を`"n; ; ; "`の形で返す。動詞は、待ってから次を呼べという`wait`、カメラ係に問い合わせてその答えを次の入力にせよという`eye`、すぐ次を呼べという`next`、完了を表す`done`の四つである。作業の状態（いまの段、置いた個数、試行回数、狙う色、土台の角度など）は、すべてエージェントのフィールドにある。

港。　Mac上の港（`dock.py`、171行）は判断をしない。エージェントのソースを主へ送り、`step`を呼ぶ時計と、カメラ係への郵便係を務める。`wait`なら指定の時間だけ待ち、`eye`ならカメラ係（Mac上のPython）に問い合わせて答えを次の`step`に渡す。`step`の呼出しは、AIPLの遠隔呼出しと同じ電文（UDP、ASCII一行、同じ要求番号で再送し、受け側は要求番号ごとに高々一度実行する）を用いる。港は返された断片番号から、ソース中の印の行を求め、シミュレータのAIPLプログラム窓へ送る。窓は、エージェントがシミュレータに居るときも実機に居るときも、いま実行した断片の行を反転表示する。作業が終わると、港はエージェントの記憶を`pack()`で取り出し、次の主で`unpack()`によって渡す。本稿で運ぶ記憶は旅の記録（訪れた主と、そこで置いた個数・段数）だけである。

主を占有する間の役目。　実機のXinuは常駐するAIPLプログラムを一本しか持たない。新しいソースを載せると、それまで腕の命令を受けていたアクターも消える。カメラ係は探索の最中に腕を直接動かすため、エージェントは主に居る間、腕のアクター（/dofbot）としての役目も兼ねる。エージェントが去ると、港は元の腕のアクターを載せ直す。

次はstepのうち、立方体の上で指を閉じる断片である。

```
} else { if (st == 7) {                       // [7] grasp (or give up on this one)
  if (input == "err") {
    var r7 = arm_cmd("pose " ++ base ++ " 80 50 50 90 30 2000");
    tries = tries + 1;  st = 1;
    out = "7;wait;2250;could not lower - look again";
  } else {
    var r7b = arm_cmd("set 6 145 800");
    st = 8;
    out = "7;wait;1300;grasp " ++ r7b;
  }
```

直前の断片で港に依頼した下降の結果がinputとして届く。下ろせなかったなら姿勢を戻して探し直しの段（st = 1）へ、下ろせたなら指を閉じて1.3秒待つよう港に求める。この断片が実機で走るのか模型で走るのかは、arm_cmdの結び付け先だけで決まる。

5.3　共通のRobot Actor契約

作業レベルの契約は汎用ロボットAPIより小さく定める。表1に、提案する操作と観測可能な結果を示す。この契約はAIPLで実装し、正典の処理系で型検査と実行を通してある（7.1節）。表に挙げた名称と型は、その実装に合わせてある。ただし、7.7節で移動させたエージェントは、この契約より一段低い水準（arm_cmdの文字列命令とカメラ係への依頼）で書いてある。生成するエージェントが契約を呼べるよう、契約の各操作を両方の主が与える操作として実装することは、今後の作業である。

Table 1: シミュレーションと実機で共通にするActorインターフェース案

操作	入力	観測可能な結果
observe()	引数なし	時刻、物体の位置姿勢、ラベル、信頼度、場面の版
pick(id)	観測された物体ID	完了または種別付きの失敗、把持状態
place(zone)	名前付き目標領域	完了または種別付きの失敗、物体位置
moveSafe(pose)	範囲内の目標姿勢	経路確認後の完了または失敗
release()	引数なし	グリッパー開放、保持状態の更新
status()	引数なし	準備状態、実行中の操作、故障状態
stop()	引数なし	停止指令と応答の記録

各呼出しに要求ID、期限またはタイムアウト、種別付きの結果を設ける。結果の例として、成功（OK）、対象不明（NOT_FOUND）、到達不能（UNREACHABLE）、衝突のおそれ（COLLISION_RISK）、時間切れ（TIMEOUT）を定める。観測には撮影時刻、座標系、場面の版を付け、古い観測を現在の目標として暗黙に利用しない。状態を変える呼出しはバックエンドが順序付ける。AIPLの非同期送信と同期応答をこの契約へどう対応させるかは実装時に定める。タイムアウトした命令を暗黙に成功と扱ってはならない。場面の観測と操作の完了は、時刻とプログラム識別子を付けて保存する。

次は実装した契約のうちpickの全文である。擬似コードではなく、処理系が型検査して実行するソースそのものである。

```
method pick(id) : int @ 3 !{act, mut} {
  var r = 0;
  if (held >= 0) { r = 3; }              // すでに保持中 → COLLISION_RISK
  else {
    var k = 0 - 1;  var i = 0;
    while (i < array_len(ids)) do { if (ids[i] == id) { k = i; } i = i + 1; }
    if (k < 0) { r = 1; }                    // NOT_FOUND
    else {
```

```
      if (zs[k] > 0.32) { r = 2; }         // UNREACHABLE
      else { held = id; version = version + 1; r = 0; }
    }
  }
  reply(r);
}
```

型検査は*返信が高々一度*であることを要求する。早期にreplyして処理を続ける書き方は拒否されるため、結果を変数に集めて末尾で一度だけ返している。目標達成はメソッドの成功応答だけで判断せず、観測結果または独立した採点器から判定する。

5.4　シミュレーションモデルと検証

机の寸法、ロボットとグリッパーの形状、カメラ座標系からロボット座標系への変換、物体の形状と質量の推定、必要に応じた摩擦の推定、目標領域を用いて場面を初期化する。CGは人による視覚確認に役立つが、検証には課題に合った幾何モデルと物理・接触モデルを用いる。各プログラムについて、物体と手先の軌跡、衝突と最小距離、把持の成否、最終配置、タイムアウト、指示固有の目標条件を記録する。

検証は三段階とする。第一に、許可された呼出しと物体参照の静的検査を行う。第二に、到達可能性、関節と作業空間の限界、経路上の衝突を調べる。第三に、指示された操作順序と最終配置を確認する。接触を伴う操作をモデルが十分に扱えない場合はUNKNOWNを返し、自動的な実機投入を保留する。欠けている情報を合格として扱わない。定めた上限回数の範囲で、構造化された失敗理由をLLMに与え、修正したプログラムを再検証してもよい。初回生成と修正後の結果は別に記録する。

5.5　実機実行と安全上の制約

各試行前にアームを較正し、場面を再観測し、緊急停止手段と作業領域の設定を確認する。実機バックエンドは関節角度と直交座標の範囲を超える命令を却下し、対応する装置では速度と把持力を制限する。通信途絶や想定外の状態では停止し、操作者が試験環境を監督する。これらは実験システムに課す条件であり、シミュレーションが物理的安全を証明することを意味しない。機体、制限値、停止機構、復帰手順は最終稿で報告する。

6　実験計画

この節は今後実施するための計画であり、完了した実験結果は含まない。試行数、装置の仕様、計画からの変更は結果を見る前に確定・記録する。

6.1　装置と作業課題

一台の机上アームとグリッパー、固定カメラまたは測定済みの物体位置入力、色付きの剛体ブロック、二つの箱、積み重ね領域を用いる。最終稿には機体、制御器、カメラ、フレームレート、グリッパー、シミュレータ、物理エンジン、LLMの版、AIPL処理系の版、プロンプト、較正手順、物体寸法を記載する。カメラによる認識を利用できない場合、初期段階では測定した物体ラベルと位置を入力してよい。その簡略化を明記し、比較条件間で同じ入力を用いる。なお本稿の測定では、位置はこの簡略化のとおり測定値を用いた。手首カメラの画像から色を判定する段は実装して実機で動かしているが（7.3節）、対象物を置いた状態での正解率は未測定である。

予定している装置。　机上アームはYahboom DOFBOT（6軸、Raspberry Pi 5版）とし、サーボ制御基板を$\mathrm{I}^2\mathrm{C}$アドレス0x15で駆動する。リンク長は公開されているURDFの値で、肩の高さ$0.1075\,\mathrm{m}$、上腕$0.08285\,\mathrm{m}$、前腕$0.08285\,\mathrm{m}$、手首からサーボ5の中心まで$0.07385\,\mathrm{m}$である。指の長さは当初$0.06\,\mathrm{m}$と置いたが、実機が卓に当たった高さから約$0.083\,\mathrm{m}$と推定し直した（7.4節。いずれも未実測）。この値で平面の逆運動学を解く。手首のカメラはUVC（YUY2、等時転送、320×240、$10\,\mathrm{fps}$）である。実機バックエンドはRaspberry Pi 5上のbare-metal Embedded Xinuで動作し、AIPL処理系の正典（OCaml実装）から

UDPの遠隔呼出しでアクターを起動する。シミュレーション側は運動学の三次元模型と、接触を扱う物理エンジンの二段とする。各要素の版は最終稿で確定する。

課題は命令の長さだけでなく、必要な推論によって分ける。単一物体の移動、複数物体の指定順での移動、色別の分類、属性による選択、積み重ね、解釈の明示が必要な指示を含める。一意に採点できない曖昧な指示は、事前に決めた規則に従って明確化または除外する。例示に使った指示と、未見の指示・配置・ラベルの組合せを分離する。データの分割はプロンプト例を書く前に行う。試行数は検出力から定める。その計算と結論は6.2節に示す。

現時点の実装状況。 Actor契約はシミュレーション側で実装を終え、生成プログラムを実行して検査できる（7.1節）。実機側では、腕の駆動と手首カメラの取得、および模型との切替までが動く（7.3節）。手で書いた振り分けのエージェントは、シミュレータの主から実機の主へ移動して両方で2個とも運んだ（7.7節）。色で振り分ける作業は、実機で青と緑の立方体に対して各1回完遂し（7.4節）、二つを一回の実行で運ぶ作業も1回完遂した（7.5節）。条件を固定した反復の完遂率は、1個で0.75、2個で0.3であった（7.6節）。投稿までに埋めるべき差は三つである。第一に、*実機側*の契約が未実装であり、実機にあるのは命令文字列を$\mathrm{I}^2\mathrm{C}$の電文へ渡す単一のメソッドだけである。第二に、実機の把持は2回成功したが、成功率を述べられるほど試行していない。第三に、シミュレーション側に接触のモデルが無く、把持の成否を検証できない。後者は評価課題そのものを成立させないため最初に解決する。把持が安定しない場合は、接触の検知、グリッパーの改修、課題を押して動かす操作へ変更する、の順に対処し、採用した対処を明記する。これらが未解決のまま数値を報告することはしない。

運動学だけの段では接触を検証できない。 このことは見込みではなく測定である。 既存の運動学シミュレータで色別分類の作業を150秒（27サイクル）実行し、記録した結果は 成功27回・失敗0回、把持誤差の最大値$0.000\,\mathrm{mm}$、搬送中の追従誤差の最大値$0.000\,\mathrm{mm}$、 解放時の設置誤差の最大値$0.000\,\mathrm{mm}$、分類の正答28/28であった。 三つの誤差がいずれも厳密に零で成功率が1.00になるのは、精度が高いからではなく、把持した物体が手先に拘束されていて滑りも落下も起こり得ないからである。 すなわちこの値は測定結果ではなく、接触を計算していないことの表示である。 同じ作業論理のまま、アクターのノード割当を軸ごとの12ノードから 機器ごとの2ノード（アーム機とカメラ機）へ変更して再実行したところ、 サイクル数、成功率、三つの誤差、分類の正答、各軸の指令角の範囲はすべて一致した。 作業論理が保たれることはこれで確かめられるが、割当の違いが持つ代償 （バス競合、待ち時間、ジッタ）は現れない。時間と競合のモデルを持たないためである。 したがって本稿では、このシミュレータを幾何の段（到達可能性、関節と作業空間の限界、 経路上の衝突）に限って用い、接触を伴う判定には`UNKNOWN`を返させて 自動的な実機投入を保留する。把持と接触の合否を検証するには、接触モデルを持つシミュレータを加える必要がある。これは本稿の範囲外であり、未実施である。

6.2 試行数と検出力

主要比較は同一の指示に対する条件Aと条件Bの対応のある二値結果であるから、検定に寄与するのは両条件の結果が食い違う指示（不一致対）だけである。p_{10}を条件Bだけが成功する指示の割合、p_{01}を条件Aだけが成功する指示の割合とし、両側$\alpha = 0.05$のMcNemar正確検定を仮定して検出力を計算した。計算手続きは公開する解析スクリプトに含める。

表2に指示数と検出力を示す。指示20件では、事前選別が実機の失敗を25ポイント減らす場合でも検出力は0.196にとどまる。この標本数では、効果が無いのか標本が足りないのかを区別できない。

Table 2: 指示数と検出力（McNemar正確検定、両側$\alpha = 0.05$）

指示数n	p_{10}（Bだけ成功）	p_{01}（Aだけ成功）	検出力
20	0.30	0.05	0.321
20	0.25	0.05	0.196
20	0.20	0.05	0.095
60	0.25	0.05	0.797
80	0.25	0.05	0.908
100	0.20	0.05	0.837

Table 3: 検出力0.80に必要な指示数

p_{10}	p_{01}	必要な指示数n	不一致対の期待数
0.30	0.05	46	16
0.25	0.05	62	19
0.20	0.05	92	23
0.15	0.05	168	34

以上から、計画値を*異なる指示60件（六つの課題群に各10件）と配置2通り*とする。想定する効果$p_{10} = 0.25$、$p_{01} = 0.05$のもとで検出力は0.797である。配置は指示内の反復であり独立した標本ではないから、主解析は指示を単位とした再標本化で行い、同じ指示の反復を独立した標本として数えない。

この計画値は確定値ではない。p_{10}とp_{01}は予備実験の結果から見積もり直し、表3を引き直して試行数を確定する。効果が想定より小さい場合は、課題群を減らして指示数を増やすか、主張を差の区間推定に変える。いずれの判断も本実験の結果を見る前に行い、変更の日時と理由を記録する。結果を見てから試行を追加することはしない。

実機の作業量は次のように見積もられる。条件Aは120試行（60指示$\times$2配置）、条件Bは合格した候補のみで、投入率を7割と見込めば約84試行、合わせて約204試行である。1試行あたり較正、場面の再観測、初期配置への復帰を含めて1.5分とすると約5時間であり、やり直しを含めて実働3日を要する。これは見積もりであって測定値ではない。

6.3 比較条件と公平性

主要な比較では*同一の初回生成プログラム*を二条件に対応させる。（A）コンパイルと静的検査の後、シミュレーションによる作業検証を行わず、制約付きで実機実行する。（B）同じ検査に加えてシミュレーションで選別し、合格した候補だけを実機で実行する。ハードウェア側の安全制約は両条件で有効にする。指示、初期配置、乱数、機体、知覚入力、成功判定を揃える。Bで却下した候補も、指示単位の全体成功率と却下率の分母に含める。重大な安全制約に違反する候補はAでも実行しない。したがって、この比較が推定するのは共通の安全ゲートに*追加した*シミュレーション選別の効果である。

副次条件として、回数を固定したシミュレーションの失敗情報による修正と、人手で正しさを確認した作業プログラムの両バックエンドでの実行を設けてもよい。前者は修正の費用と便益、後者は言語変換の失敗とSim-to-Realの不一致を区別するためである。可能な範囲で作業順と条件順を無作為化し、物体位置と較正を同一手順で戻す。操作者の介入はすべて記録する。Aでの対応試行が機体を損傷するおそれがある場合には実行せず、選択による比較上の限界を明示する。

別の副次比較では、Code as Policies型の手法として、同じ作業プリミティブを呼ぶPythonプログラムを生成する[2]。LLMの版、例示数、場面の観測、ロボットの操作、安全ゲート、試験集合を揃える。構文とAPIの妥当性、作業完了、プログラム量、修正の労力を比較し、表現力やプロンプト量の差が残る場合は報告する。これは原論文の全機能の再現を意味しない。比較を実装できない場合、AIPLの優位性に関する比較主張を削除する。

6.4 評価項目と解析

主要指標は指示単位の実機作業成功率とし、シミュレーションで却下した指示は実機で作業未完了として数える。これとは別に、投入した試行に限った成功率、投入率、危険・実行不能プログラムの却下率、衝突や接近の件数、誤った合格判定の比率、検証の追加時間を含む指示から最終状態までの所要時間中央値を報告する。失敗を構文・型、無効な参照、作業論理、シミュレーションでの到達不能・衝突、知覚、制御器、Sim-to-Realの不一致に分類する。対応するActorメッセージと場面状態の差も記録する。初回と修正後のプログラムは区別する。

却下と合格の突き合わせ。　条件Bは指示ごとに投入か却下かを決め、条件Aは同じプログラムを実機で実行する。したがって条件Aは、条件Bの判定に対する参照となる。両者を突き合わせて四つに分ける。(i) Bが投入し実機で成功、(ii) Bが投入し実機で失敗（*誤った合格*）、(iii) Bが却下したがAでは実機で成功（*誤った却*

下）、(iv) Bが却下しAでも実機で失敗（正しい却下）。誤った合格の比率は(ii)を投入数で割った値、誤った却下の比率は(iii)を却下数で割った値とする。この対応づけにより、RQ3の「どの種類を除外できたか」とRQ4の「合格しても失敗した頻度と原因」を同じ試行集合から測れる。却下は段（構文・型、Actor利用規則、幾何、物理）ごとに帰属させ、段別の内訳を(iii)と(iv)に分けて報告する。安全ゲートが条件Aで停止させた試行は、実行しなかったものとしてではなく条件Aの失敗として数え、その件数を別に報告する。条件Aでの実行が機体を損傷するおそれから見送った試行は、参照が得られない指示として(iii)(iv)の分母から除き、その件数と理由を明記する。

対応のある二値の結果について、指示を単位としたクラスタ・ブートストラップ（10^4回の再標本化）による成功率差の信頼区間と、McNemarの正確検定を示す。主解析の単位は指示であり、配置はその内部の反復として扱う。課題の群ごとの結果、位置誤差、操作時間の分布も示す。所要時間は中央値に加えて分布を図示し、平均だけを示すことはしない。少数の長引いた試行が条件間の差を隠すためである。同じ指示を反復した試行は独立した標本として扱わない。作業一覧、プロンプト、シミュレータの設定、合格閾値、除外規則、匿名化した実行履歴を、機器とソフトウェアのライセンスが許す範囲で公開する。

7　実験結果

主実験（条件Aと条件Bの対応比較）は実機を要するため未実施である。本節には完了した七つの実験を記す。7.3節から7.6節までは、以前の遠隔呼出し方式（プログラムをMacの正典処理系で動かし、腕のアクターの宛先を模型と実機で替える）で行った。最後の7.7節が、本版の方式による実験である。指示からAIPLを生成する段の予備実験（7.1節）、接触を解く検証段を加えた比較（7.2節）、同一の作業手順を模型と実機の双方で実行した記録（7.3節）、色で振り分ける作業を同じプログラムで模型と実機に実行し、実機で完遂するまでの記録（7.4節）、二つの立方体を一回の実行で運ぶ作業の記録（7.5節）、条件を固定した反復による実機の基礎性能（7.6節）、およびモバイルエージェントとしての移動実行（7.7節）である。

7.1　生成の予備実験（RQ1）

装置。　表1のActor契約を実際にコンパイルできるAIPLとして実装し、シミュレーション・バックエンドとして机上の場面（剛体5個、目標領域3つ）を持たせた。物体は`[id, 色, 形, x, z]`で観測され、1個は作業空間の外（到達不能）に置いてある。生成器は`claude-opus-5`である（温度指定はこのモデルでは受け付けられないため用いていない）。プロンプトに入れたのは、契約のソース、結果コードの定義、使用してよい言語機能、および手本2件だけである。手本は*開発用に分けた指示*からのみ取った。指示集合の分割はプロンプトを書く前に確定し、記録してある。

指示。　課題の群ごとに、未見の指示を5件ずつ、計30件用意した。群は、単一物体の移動（G1）、複数物体の指定順での移動（G2）、色別の分類（G3）、属性による選択（G4）、積み重ね（G5）、解釈の明示が必要な指示（G6）である。G6には、対象が一意に定まらないもの、観測に存在しない属性を要求するもの、到達不能な物体を指すものを入れた。例を挙げる。

> G1「緑の立方体を右の箱に移してください。」
> G3「赤を左の箱、青を右の箱、緑は積み重ね領域に分けてください。」
> G4「赤い物体のうち、立方体のほうを右の箱に入れて。」
> G6「一番大きい物体を右の箱に入れて。」（大きさは観測に含まれない）

生成物。　G4の例に対して生成されたプログラムをそのまま示す。手を加えていない。

```
// 指示: 赤い物体のうち、立方体のほうを右の箱に入れて。
class Task {
  method run(r: RobotActor) : int @ 2 !{act, mut} {
    var done = 0;
    var s  = now r.observe() timeout 500 else [0, []];
    var os = s[1];
    var target = 0 - 1;  var count = 0;  var i = 0;
    while (i < array_len(os)) do {
      var o = os[i];
```

```
      if (o[1] == "red") { if (o[2] == "cube") { target = o[0]; count = count + 1; } }
      i = i + 1;
    }
    if (count == 1) {                       // 一意に定まるときだけ動く
      var a = now r.pick(target) timeout 3000 else 4;
      if (a == 0) {
        var b = now r.place("right bin") timeout 3000 else 4;
        if (b == 0) { done = 1; }
        else { var c = now r.release() timeout 1000 else 4; }
      }
    }
    reply(done);
  }
}
```

生成器は観測から対象を選び、該当が一意であることを確かめてから動かし、各結果コードを検査している。作業論理だけが書かれており、逆運動学、サーボ指令、座標変換は現れない。役割の境界（5節）が保たれている。

結果。　表4に示す。取得できた28件のうち27件が構文・型検査を通過した。不合格の1件は型の誤りではなく、コード以外の文字列がプログラムの先頭に混入したための構文エラーである。規約違反（余分なクラス定義、入口の型の相違）は他に無かった。指示単位の目標達成は27/30であり、分母には取得できなかった2件を含めてある。

Table 4: 生成の予備実験（未見の指示30件、`claude-opus-5`）

群	内容	取得	型検査通過	目標達成	分類器の拒否
G1	単一物体の移動	4/5	4/4	4/5	23
G2	指定順での移動	4/5	3/4	3/5	26
G3	色別の分類	5/5	5/5	5/5	19
G4	属性による選択	5/5	5/5	5/5	0
G5	積み重ね	5/5	5/5	5/5	0
G6	解釈の明示が必要	5/5	5/5	5/5	3
計		28/30	27/28	27/30	71

曖昧な指示の扱い。　G6の5件はすべて、当て推量で動かさずに何もしなかった。たとえば「適当な物を適当な箱に入れて」に対しては、対象と目標領域のいずれも一意に定まらないと注記したうえで、何も動かさずに0を返している。到達不能な物体を指す指示では、その物体を選んだうえで`pick`を呼び、`UNREACHABLE`を失敗として扱っている。目標達成の判定はメソッドの応答ではなく、実行後に観測した各領域の個数から行った。

生成の段は純粋な関数ではない。　30件を得るのに99回の要求を要し、そのうち**71回が安全分類器に拒否された**（`stop_reason`が`refusal`、種別は`cyber`）。内容は机上の把持と配置だけであり、拒否は誤検出である。同一の指示が拒否されたり通ったりするため決定的でもない。2件は14回試みてすべて拒否され、取得できなかった。したがって生成の成否を報告するときは、**要求回数と拒否回数を併記しなければならない。**黙って再試行すると、生成器の能力と配信系の挙動が区別できなくなる。この現象は本稿の提案方式とは独立であり、LLMを実験の構成要素として用いる研究一般に関わる。

この予備実験の限界。　場面は1種類、バックエンドはシミュレーションのみ、生成器は1種類、プロンプトも1種類である。ここで測れたのは、契約を与えたときに検査を通る作業プログラムが得られるか（RQ1）までであって、実機での実行可能性ではない。標本数も本実験の計画値（6.2節）には満たない。

7.2　接触を解く検証段の追加（RQ3）

装置。　7.1節の契約に、二つ目のバックエンドを実装した。第一段（以下Tier-1）は運動学だけを解き、把持した物体を手先に拘束する。第二段（以下Tier-2）は剛体接触を解く物理シミュレータ（MuJoCo 3.14）で、**把持した物体を手先に拘束せず、指と物体の摩擦だけで保持する。**リンク長は実機の実測値（肩高0.1075、上腕と前腕0.08285、手首0.07385、把持点$+0.06$ m）を用い、順運動学が実機の式と一致することを6姿勢で確かめた（最大誤差0.0000 mm）。アームはトルク駆動に重力補償を加えて制御する。

把持の成否は観測で判定する。　持ち上がったかどうかという幾何だけの判定では、弾き飛ばした物体がたまたま手先の近くにある場合に「掴めた」と誤る。そこで把持の直後に手先を12 mm横へ動かし、手首カメラの中で像が動かないことを確かめる。掴めていれば物体はカメラとともに動くので像は不動、掴めていなければ物体は机に残って像がずれる。実測では、掴めている場合の重心移動が0.0 px、掴めていない場合が2.5–6.8 pxで、明確に分かれた。

同一ソースの両実行（RQ2）。　生成されたAIPLは*一行も変えていない*。変えたのは契約の実装だけであり、Tier-2版は同じ名前・同じ引数・同じ結果コードを接触計算の結果として返す。処理系そのものには手を入れず、実行時に組込み関数を差し込んで接続した。7.1節で得た27件はいずれも両バックエンドで実行できた。

結果（RQ3）。　表5に示す。Tier-1は27件すべてを合格とし、Tier-2は23件を合格とした。差の4件はすべてTier-1が見逃したもの、すなわち*誤った合格*である。逆向きの食い違い（Tier-1が落としTier-2が通す）は無かった。

Table 5: 同一の生成プログラム27件を二つの検証段で実行した結果

群	件数	Tier-1 合格	Tier-2 合格	誤った合格
G1 単一物体の移動	4	4	4	0
G2 指定順での移動	3	3	2	1
G3 色別の分類	5	5	5	0
G4 属性による選択	5	5	5	0
G5 積み重ね	5	5	2	**3**
G6 解釈の明示が必要	5	5	5	0
計	27	27	23	4

誤った合格はG5（積み重ね）に集中し、3件はいずれも*3個目*で失敗した。実行履歴では3個目の`place`が到達不能を返している。原因は7.3節で述べる実機の可達域の性質——高さが増すほど届く範囲が狭まること——であり、高さに依存しない可達判定しか持たないTier-1では再現できない。

有利な結果が出たときこそ原因を追う。　最初の実行ではTier-2の不合格は6件だった。原因を1件ずつ追ったところ、うち3件は物理現象ではなく**目標領域に壁を作っていなかったこと**によるものだった。置いた物体が次の物体に押し出され、領域の外へ転がっていたのである。壁を加えると3件とも合格に変わった。追跡しなければ、モデルの不足を手法の成果として報告していた。本稿の数値は、個別に原因を追跡し、さらに把持の高さと移動経路を修正したうえでの4件である。再現性は確認しており、設定を変えずに実行した3回は27件すべてで同一の判定であった。

この比較の限界。　場面は1種類、機体は1種類、物体は剛体の直方体のみである。摩擦係数と接触剛性は実測ではなく、妥当と思われる値を置いた。ここで示せたのは*接触と高さ依存の可達を加えた検証段が、加えない検証段の見逃しを検出し得ること*までであり、Tier-2の合否が実機の成否をどれだけ予測するかは測っていない。それには対応する実機試行が要る。

7.3　同一の作業手順の模型と実機での実行（RQ2）

装置。　実機はYahboom DOFBOT（6軸）で、Raspberry Pi 5の上でbare-metalのEmbedded Xinuが動き、RP1のI²Cでサーボ基板（アドレス`0x15`）を駆動する。手首にはUVCカメラ（320×240、10 fps）が付く。模型はブラウザ上のXinuデスクトップ（aice-avm）に含まれる運動学の腕で、同じ文字列命令を受ける。両者は同一の窓から切り替えられ、*命令の列は一行も変えない*。命令は`pose a1..a6 ms`と`set id ang ms`である。

手順。　立方体一個を運ぶ最小の作業を10手で書いた。原点、グリッパ開、接近姿勢、下降、把持、持ち上げ、旋回、下降、解放、原点復帰である。姿勢はすべて実機のサーボ可動域0–180°に収まることを実行前に機械的に検査した。

結果。　模型では4回の実行すべてで10手が`ok`を返し、読み戻しも指令どおりであった。実機でも同じ手順が動き、最終姿勢が指令と1°以内で一致した実行がある。すなわち*同じ手順が両方の実装で動く*。ただし実機では応答の欠落が生じた（次段落）。

模型と実機で異なった点。　二つある。いずれも模型には現れない。

第一に、**実機は応答を落とす**。同じ手順を4回実行し、計40手のうち**9手（22.5%）で応答が返らなかった**。サーボ基板はサーボ通信中にI²Cの応答が20 msを超えることがあり、書き込みは受理されるが読みが落ちる。模型では40手すべてが応答し、この事象は一度も起きていない。

ここで重要なのは、**応答の欠落が指令の不達を意味するとは限らない**ことである。9手のうち7手は指令が実行されており、続く手は正常に応答した。残る2手は指令自体が届かず、腕は前の姿勢のまま止まった。すなわち*応答が無いという一つの観測に、二つの異なる状態が対応する*。応答の欠落を失敗と解釈すれば、実行された作業を不必要に中断する。成功と解釈すれば、実行されていない指令を成功と数える。どちらの解釈も22.5%の頻度で誤る。

著者自身もこの区別を誤った。ある実行で最終姿勢が指令と一致しないのを見て実機の異常と判断したが、実際には1.8秒かけて移動している途中を読んだだけであった。**状態の読み取りには、動作の完了を待つか、指令に紐づく識別子が要る**。要求IDと期限を持つ契約（5節）が必要になるのはこのためであり、本稿の実装はまだそれを備えていない。

第二に、**実機の可達域は模型より狭い**。サーボが0–180°であることは、関節角jとの対応（$s_1 = 90 + j_1$、$s_2 = 90 - j_2$、$s_3 = 90 - j_3$、$s_4 = 90 - j_4$）を通じて$j \in [-90^\circ, 90^\circ]$を意味する。この制約の下で、工具をほぼ鉛直にしたときに届く水平距離を計算すると表6のようになる。

Table 6: 実機のサーボ可動域0–180°の下で届く水平距離（工具傾き10°）

手先の高さ	実機の制約あり	制約なし（幾何だけ）
$z = 0.0155$ m（把持）	$r = 0.134$–0.183 m	$r = 0.020$–0.182 m
$z = 0.040$ m（一段目を置く）	$r = 0.122$–0.170 m	$r = 0.020$–0.175 m
$z = 0.058$ m（二段目を置く）	$r = 0.107$–0.152 m	$r = 0.020$–0.165 m
$z = 0.076$ m（三段目を置く）	**可達域なし**	$r = 0.020$–0.155 m

制約を入れると可達域は殻状になり、内側が失われる。近い対象に届かないのは肘の曲げが90°を超えてサーボ範囲を外れるためで、肘の解を上下とも試しても$r < 0.134$ mでは範囲内に収まらない。とくに**三段目を置く高さには、机上のどこであっても届かない**。すなわち、この機体では立方体を三段に積む作業は原理的に実行できず、7.1節で生成されたG5群の指示のうち三段を求めるものは、プログラムが正しくても達成され得ない。幾何だけを見る検証では、この不能性を検出できない。

手首カメラ。　実機のカメラからRGB565の画像を取得し、7.1節と同じ色の判定器（画素を明るさで正規化した色度にし、既知の参照色のうち最も近いものへ割り当てる）を適用した。机に対象物を置いていない状態では、判定器は`None`を返し、どの色にも割り当てなかった。対象が無いときに当て推量をしない点は意図どおりである。対象物を置いた状態での正解率は未測定である。

この実験の限界。　作業は一個の運搬、手順は一通り、試行は4回である。机上に対象物を置いていないため、把持の成否は測っていない。ここで示せたのは*同じ手順が両方の実装で動くこと*（RQ2）と、*模型に現れない差が二つあること*までである。シミュレーションによる事前選別が実機の成否をどれだけ予測するか（RQ4）は測っていない。

7.4　色で振り分ける作業の実機での完遂（RQ2・RQ3）

（本節から7.6節までは以前の遠隔呼出し方式による。）7.3節の手順は姿勢を手で書き、把持の成否を測っていなかった。そこで、立方体を*探して*挟み、色によって左右の置き場へ運ぶ作業を一つのAIPLプログラムに書き、同じプログラムを模型、実機の順に実行した。2026年9月26日から29日までの4日間に、このプログラムを20回実行した。実機が立方体を挟んで置き場へ運んだことを目視で確認できたのは最後の2回で、青と緑の立方体でそれぞれ1回である。本節の数値はすべてこの20回の実行記録から数えたもので、統計的な評価ではない。

構成。　AIPLのプログラムは、腕へ命令を送る`Dancer`、カメラ係へ問い合わせる`Eye`、手順を組む`Planner`の三つのアクターからなる。模型と実機で異なるのは、腕のアクターの宛先（模型は`127.0.0.1:9010`、実機は板の`192.168.3.101:9010`）と、カメラ係へ渡す`sim`／`real`の一語だけである。カメラ係はMac上のPythonで、AIPLの遠隔呼出しと同じ電文で`locate`（探す）、`lower`（下ろす）、`held`（挟めたか）に答える。模型にはCGの立方体（掴む、運ぶ、置くに追従する）と、手首カメラと同じ位置・画角から描いたCGの画像を加えた。色の判定は、模型ではCGの画素、実機ではXinuのカメラの画素に、*同じ判定器*を適用する。腕とカメラの寸法は一つのファイルに置き、CG、逆運動学、画像から卓上への逆射影がすべてそれを読む。各寸法には出典（URDF、仮置き、実測）を付けた。

手順。　(1)カメラの軸が立方体の中心を通る姿勢を逆運動学で求めて撮り、青または緑の画素の重心を卓上へ逆射影して位置を推定する。推定位置へカメラを向け直し、移動が6 mm未満になるまで繰り返す。(2)手首のカメラは指の軸から3.2 cm上に付いているため、撮った後に指の軸を立方体へ合わせた姿勢へ移り、卓上5 cmから0.5 cmずつ下ろす。(3)指を閉じて持ち上げ、挟めたかを確かめる。挟めていなければ探し直す（上限30回）。(4)緑は右（土台$150°$）、青は左（土台$30°$）の置き場へ同じ方法で下ろして放す。

結果。　19回目の実行で、模型は緑の立方体を、実機は青の立方体（1辺3 cm）を、いずれも1回目の試行で挟み、指定の置き場へ置いた。20回目は新しい緑の立方体で行い、実機は6回目の試行で挟んで右の置き場へ置いた（後述）。19回目では、実機でプログラムが腕へ送った10手はすべて応答し、腕の基板は最後まで応答を保った（探す・下ろす段の細かな動きはカメラ係が送り、この10手には含まない）。20回の実行で明らかになった模型と実機の差を表7に示す。いずれも運動学の模型では検出できず、実機を動かして初めて分かったものである。

「挟めた」の判定。　当初は、持ち上げた後に卓上を探し直し、立方体が見えなければ挟めたとしていた。この判定は5回「挟めた」と答え、そのうち少なくとも4回は、実際には運べていなかった（目視による報告、または下ろす段が失敗していた記録による）。立方体が押されて視野の外へ出ても、見えないという同じ観測になるためである。これは7.3節の「応答が無い」と同じ構造で、*一つの観測に異なる二つの状態が対応する*。そこで判定を指のサーボの読み戻しに替えた。指令135（幅2.8 cm）で閉じたとき、3 cmの立方体を挟んでいれば指は幅3.0 cm（指令131）付近で止まり、空なら135まで閉じる。読み戻しが133以下なら挟めたとした。替えた後は7回判定した。挟めたとした2回は、いずれも置き場へ運んだことを目視で確認した。挟めていないとした5回は、いずれも読み戻しが135で、指が閉じ切っていた。ただし成功した2回の読み戻しはいずれも133で、しきい値と等しく、余裕は無い。

取り違え。　20回目の1・2・5回目の試行では、判定した範囲に占める青の画素が1.7–7.7%しかない小さな塊を立方体とみなして挟みに行き、空振りした。緑の立方体を見つけたのは6回目の試行で、範囲の26.8%が緑であった。3・4回目の試行では何も見つからなかった。空振りの3回は、いずれも指の読み戻しが135で、正しく挟めていないと判定されて探し直しに戻った。模型の卓上には立方体しか無いため、この取り違えは

Table 7: 色で振り分ける作業で見つかった模型の前提と実機の差（20回の実行から）

項目	模型の前提	実機で分かったこと
カメラ画像の向き	$180°$回転（Linuxで撮った時の記録）	回転していない。土台と腕を動かし、立方体の写る位置が動く向きを実測した
カメラの取付面	手首の下面	上面。位置の推定が逆向きに狂っていた
指の開き（指令→幅）	$30 \to 5$ cm、$135 \to 1$ cmの直線	物差しで7段を実測：$30, 50 \to 6.2$、$70 \to 5.8$、$90 \to 5.2$、$110 \to 4.2$、$131 \to 3.0$、$135 \to 2.8$ cm。指令110では3 cmの立方体に触れていなかった
指の長さ	6 cm（仮置き）	模型の計算で指先を卓上2.3 cmに下げると実機は卓に当たり本体が浮いた。約2.3 cm長いと推定し8.3 cmとした（未実測）
卓との接触	関節角の読み戻しで検出できる	本体が浮くだけで関節は指令どおりに回り、読み戻しでは検出できない
土台の向き	計算どおり	実機は約$1°$左を掴みに行く。$+1°$を補正した
立方体を見られる範囲	前方に広い	指を8.3 cmとすると、腕の最下点を卓上4 cm以上に保って撮れるのは土台の軸から14–23 cmに限られる
卓上の他の物	立方体だけ	青いケーブルなどの小さな色の塊を立方体と取り違え、3回空振りした（20回目）

模型では起こらない。対策として、画像の1.5%未満の色の塊を立方体とみなさないようにした。3 cmの立方体は見る距離で画像の5%以上に写る。この対策の効果は未検証である。

実機の運用で起きたこと。 腕の基板は、電源を入れて数分の動作で応答しなくなることが4回あり、電源を入れ直すまで戻らなかった。止まる直前には、Xinu側の腕の処理が送る基板リセットが記録されていた。この処理は、6軸とも角度が読めない状態が3回続くとリセットを送る。サーボの動作中は読み取りが失敗しやすいため、動作直後に読み取りを重ねるとこの条件に当たる。読み取りを2回まで、間隔1.5 秒に制限した後の2回の実行では、基板は止まらなかった。しかし、Xinu側の自動リセットそのものを外したカーネル（9月29日）でも、基板は再び止まった。したがって自動リセットは原因ではなく、止まる原因は特定できていない。また、ブラウザが板のHTTPへ要求を送らない接続を張ったままにすると、単一スレッドのHTTPが全体として止まった。これは3回観測し、いずれもブラウザを終了すると直ちに回復した。

この実験の限界。 実機で作業を完遂したのは2回（色の異なる立方体で各1回）で、成功率を述べる標本ではない。指の長さは推定値のままで、挟めた判定のしきい値には余裕が無い。20回の実行の間に、プログラム、カメラ係、寸法を何度も直している。そのため、各回は同じ条件の反復ではない。ここで示せたのは、*同じAIPLプログラムが模型と実機の両方で作業を完遂し得ること*（RQ2）と、*運動学の模型が見逃す差が、実機を動かす過程で八つ見つかったこと*（RQ3の見逃しの具体例）までである。事前選別が実機の成否をどれだけ予測するか（RQ4）は、依然として測っていない。

7.5 二つの立方体を一回の実行で運ぶ（RQ2・RQ3）

7.4節のプログラムを、卓上の緑と青の立方体を*両方とも*置き終えるまで「探す、挟む、色の置き場へ置く」を繰り返す形に拡張した。1個置いたら残りの色だけを探し、手の届く範囲を一巡して見つからなければ終える。2026年9月29日に15回実行した（2個を置いた実行が13回、途中で1個に戻した実行が2回）。実機が2個とも置き場へ運んだことを目視で確認できたのは最後の実行である。本節の数値はこの15回の記録から数えた。

探し方。 7.4節の探索は、毎回同じ所から撮り始め、最初に見えた色の塊を追っていた。この方法は、2個目や少し外れた位置の立方体を繰り返し見落とした。そこで、手の届く帯（土台の軸から14–23 cm、正面から$\pm 45°$）を、カメラの写る範囲（見る距離13 cmで横15 cm、奥11 cm）が重なるように2列×4か所に分け、

決まった順に全部撮って、緑と青の塊を卓上へ逆射影した一覧を作るようにした（一巡に約54秒）。腕が移動している間に取れた画像にも塊を探し、一巡で探す色が見つからなければその位置へ戻って見直す。向きを変えても画像の同じ位置に写る塊は、卓上の物ではなくカメラとともに動く物として除いた。

結果。　最後の実行で、模型は緑と青をそれぞれ1回目の試行で運んだ。実機は3回の試行で2個とも運んだ。1回目は青を空振りし（指の読み戻し143、持ち上げ後145、すなわち閉じ切った）、2回目で青を、3回目で緑を挟んだ。挟んだ2回は、閉じた直後と持ち上げた後の読み戻しがいずれも139で、運ぶ途中で落としていない。この間、腕の基板と板のHTTPは止まらなかった。

15回の実行で見つかったこと。　表8に示す。模型に無いもの（卓上の他の物、照明と露出、指のたわみ、ブラウザと板の通信）が、いずれも実機で初めて現れた。

Table 8: 二つの立方体を運ぶ作業で見つかった問題（2026年9月29日の15回の実行から）

問題	実機で起きたこと	対処
卓上の他の物	卓の水色のシートを青と判定し、最初に見た色に固定して緑を見落とした	青は彩度0.8以上（シートは0.3–0.7、立方体は1.0）。置いた後は残りの色だけを探す
照明と露出	白い紙の上でカメラの露出が下がり、緑の立方体が明るさ0.1に写って判定から外れた	緑は明るさ0.06以上で判定
指のたわみ	3 cmの立方体を挟むと指の読み戻しは139で、無負荷の開き（131で幅3.0 cm）から決めた判定範囲125–134を外れた。挟めているのに「挟めていない」とし、プログラムが自ら指を開いて落とした	閉じ切れずに止まったこと（読み戻し120–141、空なら144–145）で判定
偽の成功	2回の実行が「2個置いた」と報告したが誤り。探索が問い合わせの期限（60秒）を超えて返した`err`を色として扱った回と、開いた指（読み戻し40）を「挟めた」と数えた回	期限を3分に延ばし、`err`を「見つからない」と同じに扱う。開いた指は数えない
板のHTTP	ブラウザが板へ要求なしの接続を張ったまま、またはカメラ係の問い合わせが重なると、単一スレッドのHTTP全体が止まった	板への問い合わせをカメラ係の1本の列にし、ブラウザは係から画像を読む
模型の停止	窓が背面にあるとブラウザがタイマーを間引き、模型の腕が途中で止まって古いCG画像が送られた	模型の刻みをWeb Workerから与える
腕の基板の停止	Xinu側の自動リセットを外した版でも、動作中に基板が応答しなくなった（1回）	未解決。原因を特定できていない

表のうち「指のたわみ」と「偽の成功」は、*実機の状態を確かめる判定そのものの誤り*である。模型では指がたわまず、判定は常に正しく働くため、これらは模型の実行からは見つからない。挟めたかの判定は7.4節で「見えない＝挟めた」の誤りを除くために導入したものであるが、判定の閾値を無負荷の測定から決めたことで、今度は反対向きに誤った。実機の状態を読む判定は、それ自体を実機の負荷の下で検証する必要がある。

この実験の限界。　2個を運ぶ作業を完遂したのは1回で、成功率を述べる標本ではない。15回の実行の間にプログラム、カメラ係、判定の閾値、Xinuのカーネルを直しており、各回は同じ条件の反復ではない。模型が15回のうち3回失敗したのは、いずれもカメラ係と窓の実装の誤り（変数名の衝突、寸法の読み込み失敗、タイマーの間引き）による。

7.6　実機の基礎性能（E0′）

主実験の前に、条件を固定した反復で実機の完遂率を測った。手順書（試行の順番、成功の定義、合格条件）は、結果を見る前に固定した。立方体1個を5か所（土台の軸から15–20 cm、正面から0°–±35°）に色2

種で置き、20回実行した。合格条件は、完遂率0.8以上（20回中16回以上）、かつ腕の基板の停止が1回以下とした。物体は操作者が置いた。置く位置は、腕の指先で指し示した真下とした。成功は、置き場を撮った画像と目視で判定した。

結果。　完遂は**20回中15回（0.75）**で、合格条件に届かなかった。腕の基板は、無効とした1回を含む実行中に**3回**止まり、これも条件を満たさなかった。位置別の完遂は、正面17 cm（P1）が4/4、手前15 cm（P2）が2/4、20 cm・20°（P3）が3/4、17 cm・35°（P4）が4/4、20 cm・35°（P5）が2/4であった。成功した15回は、いずれも最初の把持で挟めた。所要時間は中央値2.10分（1.77–2.97分）であった。指の読み戻しは、挟んだ場合が138–140、空振りが143–145で、閾値141によって全件が正しく分かれた。

失敗の分類。　5回の失敗は三種に分かれた。(1)*手前の青を見落とした*（2回、いずれもP2の青）：青の立方体は彩度1.0に写ったが、白い紙の上で露出が下がり明るさは0.1–0.2であった。水色のシートとの取り違えを防ぐために設けた明るさの下限0.2を下回り、判定から外れた。(2)*腕の基板の停止*（2回）：挟む動作の最中に基板が応答しなくなった。(3)*奥での空振り*（1回、P5）：指が立方体の奥に下りた。操作者は、試行中に卓が滑った可能性を報告した。

途中で直したこと。　20回の間に、振り付けや判定の閾値は変えていない。ただし、周辺の不具合を4件直した。(i)板への問い合わせを毎回RSTで切るようにした。時間切れで閉じた接続を、XinuのHTTPが待ち続けていたためである。(ii)実機の画像の時刻を、取り終えた時刻から板が写し取った時刻に改めた。腕が着く前の画像を使い、置き場の確認が左右逆に出ていたためである。(iii)画像の大半に広がる色の面は立方体とみなさないようにした。置き場の横の緑の本を、緑の立方体と判定したためである。(iv)Macのアドレスが変わった際に、カメラ係が問い合わせを控えるようにした。いずれも手順書に日時と理由を記録した。

失敗した試行のやり直し。　20回の終了後、失敗した5試行を、主任研究者の指示でやり直した。やり直しのうち3回は、失敗の分析から決めた二つの変更の後に行った。変更Aは青の明るさの下限を0.06に下げること、変更Bは挟めたと確かめた後に指の指令を145から140へ緩めて運ぶことである。Bは、押し当てたままの電流が基板を止めるという仮説に基づく。5試行のやり直しに7回実行し、5試行とも最後には完遂した。試行12（P2の青）は、変更の前の1回目で同じく見落として失敗し、2回目は実行中にMacのアドレスが変わって板との通信が止まったため無効とし、変更の後の3回目で完遂した。したがって有効なやり直しは6回で、完遂5回である（変更の前3回中2回、後3回中3回）。変更の後の実行では、基板は止まらなかった（3回）。失敗をやり直しで置き換えれば20/20になるが、*これは結果を見た後の置き換えであり、条件も混在するため、成功率の推定には用いない*。変更AとBの効果は、変更後の条件で最初から20回やり直して測る。

2個の試行。　続けて、緑と青を同時に置く試行を10回行った（位置の組5通り×2回）。条件は変更A・Bを入れたものとした。完遂は**10回中3回（0.3）**で、合格条件の0.7に届かなかった。失敗7回の内訳は次のとおりである。腕の基板の停止が4回あった。位置の計算の不具合が2回あった。一つは、内側の端で位置を詰める途中にあきらめたもの、もう一つは、挟む姿勢の手の傾きを5°刻みで探していたため、13.6–14.2 cmで姿勢が作れなかったものである。空振りで立方体を押し出したのが1回であった。10回の途中で条件を3回変えた（詰める段で14.1 cmにとどめる、下ろす処理を1回の指令にする、手の傾きを1°刻みで探す）。成功した3回は、いずれも後半に集中した。したがって、0.3は同じ条件を反復した成功率ではない。失敗した7試行は、主任研究者の指示でやり直した（8回実行し7回成功。うち1回は基板の停止で失敗）。1個と2個を合わせると、やり直しは15回実行し、無効1回を除く14回中12回で完遂した。いずれも結果を見た後の実行であり、成功率の推定には用いない。成功した回の所要時間は、中央値2.98分（2.37–8.32分、やり直しを含む10回）であった。指の読み戻しは、挟んだ場合が138–140（50件）、空振りが143–145（9件）で、閾値141はここでも全件を正しく分けた。

腕の基板の停止。　E0′の全46回の実行で、基板は8回止まった。2個の試行に限ると18回中5回である。止まった場面は、探索の最中、下ろしている最中、置いた直後とさまざまであった。原因の候補として、Xinu側の自動リセット、押し当てたままの電流、短い間隔で続く指令の三つを順に除いた。しかし、いずれの変更の後にも停止は起きた。アダプタは純正品である。停止の原因は特定できていない。

この実験が示すこと。　固定した手順での完遂率は、1個で0.75、2個で0.3であり、いずれも主実験の前提とした水準に届かなかった。失敗の主因は、照明による物体の見落とし、位置の計算の不具合、制御基板の停止であった。どちらも運動学の模型にも接触を解く模型にも含まれていない。したがって、模型の上で検証に合格したプログラムであっても、実機での完遂は、少なくともこれらの要因の分だけ下がり得る。これは、事前選別の効果を測る主実験（RQ4）で、模型の判定と実機の結果を突き合わせる際の上限を与える。

7.7　モバイルエージェントとしての移動実行（RQ2）

7.5節の作業（緑と青の立方体を探して挟み、緑を右、青を左の置き場へ運ぶ）を、5.2節のモバイルエージェントとして書き直した。ソースは174行で、断片は19個（`[0]`から`[16]`、`[90]`、`[99]`）である。探し方、下ろし方、挟めたかの判定はカメラ係が担い、7.6節と同じ設定を用いた。振り付けの姿勢と待ち時間は7.5節のプログラムと同じにした。2026年10月11日に港を4回起動した。記録は公開リポジトリ`yaskodama/dofbot_pi5`の`agent/runs/`にある。

結果。　表9に示す。4回目の起動で、エージェントはシミュレータの主で2個を運んだ後、同じソースのまま実機の主へ移動し、実機でも2個とも運んだ。実機では、探索の1回目で青を、2回目で緑を見つけ、どちらも1回目の把持で挟んだ。指の読み戻しによる判定は、閉じた直後と持ち上げた後の計4回とも「挟めた」であった。両方の主で段の数は35で一致した。到着時にエージェントが`arm_cmd("ver")`で読んだ主の名乗りは、シミュレータで`board version 0.sim`、実機で`board version 0.21`であった。エージェントが最後に返した旅の記録は`[board version 0.sim: placed 2 in 35 steps]` `[board version 0.21: placed 2 in 35 steps]`であり、シミュレータの主で書いた記録が実機の主まで運ばれている。

Table 9: モバイルエージェントの移動実行（2026年10月11日、港の起動4回）

起動	シミュレータの主	実機の主	備考
1（15:18）	2個、35段、107秒	（行かない）	シミュレータだけの確認
2（15:20）	2個、35段、107秒	移動できず	ソースを送ったが取り込まれず、エージェントが応答しなかった
3（15:22）	（2の記憶を引き継ぐ）	0個、9段、229秒	移動の後に手首カメラが止まり（USBの列挙に失敗）、3回の探索で何も見えずに終了した
4（15:29）	2個、35段、107秒	**2個、35段、144秒**	1回目の送信は取りこぼされ、港が確かめて再送した。実機の電源を入れ直した後の実行

同じソースを二つの主で動かすために揃えたこと。　二つの主は同じAIPLを名乗っていたが、同じソースを受け入れるまでに表10の差を揃える必要があった。いずれも、ソースを両方の主へ実際に送って初めて見つかった。シミュレータ側の3件は実行系を直した。実機側の3件はカーネルの焼き直しを要するため、エージェントの書き方で回避した。

以前の方式との違い。　以前の方式（7.4–7.6節）では、プログラムは常にMacの正典処理系で動き、模型と実機で替えたのは腕のアクターの宛先と`sim`／`real`の一語であった。本方式では、エージェント自身が主へ移り、腕の命令は主の中で`arm_cmd`として発せられる。宛先も`sim`／`real`の語もソースから消えた。一方で、カメラ係（知覚）と港（時計）はMacに残っている。したがって、実機の主に居るエージェントは単独では作業を完遂できず、Macとのネットワークが切れれば止まる。移動したのは作業の判断と腕の駆動であり、知覚と時間の管理ではない。

この実験の限界。　エージェントは手で書いたものであり、LLMが生成したものではない。実機で2個を運べたのは移動実行の1回で、成功率を述べる標本ではない。運んだ記憶は旅の記録だけである。シミュレーションの結果を移動の条件にする関門（式(1)）は、港の手順としてはまだ実装していない（港はシミュレーシ

Table 10: 同じソースのエージェントを二つの主で動かすために揃えた差

主	差	対処
シミュレータ	腕の基本操作`arm_cmd`が無い	命令を追加し、模型の腕に結び付けた
シミュレータ	文字列の`==`が番号で比べられ、届いた`"none"`とソース中の`"none"`が等しくならない。コンパイラも文字列の比較を拒否する	中身で比べるようにした
シミュレータ	実行中に作る文字列の置き場が256個で、再利用されない。一歩ごとに命令文字列を作るエージェントは数十歩で使い切る	同じ文字列を再利用し、上限を広げた
実機	フィールドの初期値に整数・真偽値しか書けない	文字列のフィールドは到着時の断片で設定する
実機	コメント中の非ASCII文字（§）でプログラムが黙って壊れる	断片の印をASCIIの`[n]`にした
実機	`web_expose`を二つ書くと、どちらの経路も最初のアクターに結び付く	腕の役目をエージェント自身が兼ね、公開するアクターを一つにした
運用	実機へのソースの送信が、応答を返したのに取り込まれないことがある（表9の2と4）	港が到着後に`status`を問い、答えなければ送り直す（最大3回）
運用	実機にソースを載せた後、手首カメラが止まり、再開できなかった（表9の3）。原因は特定できていない	港が実機へ移る前にカメラの配信を確かめ、止まっていれば始動し、始動できなければ移動を中止する。復旧は電源の入れ直しによった
運用	シミュレータを自身の再起動機能で起こし直すと、遠隔呼出しの受け口のスレッドが起動直後に終わる	出力をファイルへ向けて起動し直した

ョンの成否にかかわらず実機へ移る）。実機の主での段の数がシミュレータと一致したのは、今回の配置で探索の空振りが無かったためであり、一般に一致するわけではない。

［追記予定：主実験（条件A/条件B）の後に、標本数、実装の版、課題別の結果表、信頼区間、失敗の分類、所要時間、シミュレーションと実機の差を記入する。提案段階の設計から数値的な効果を推測して書かない。］

8　考察

提案方式には異なる二つの検査段階がある。コンパイル時の検査は不正なActor呼出しを除き、シミュレーションは作業上の実行不能性と一部の物理的干渉を発見し得る。実験で良い結果が得られても、それが支持するのは評価した機体、物体、場面分布における事前選別の効果である。一般的な安全性、別の機体への移行、モデルの精度を超えた接触作業への有効性までは示せない。一方、投入率を下げれば実機の失敗が減っても全体の作業完了率が下がり得るため、両方を報告する。

モバイルエージェント方式の利点は、シミュレータで実行したものと実機で実行するものが同じソースであることを、送ったバイト列そのもので確かめられる点にある。宛先や`sim`／`real`の語を替える方式では、替えた箇所が作業論理に影響しないことを別に示す必要があった。代償は二つある。第一に、主ごとの処理系の差が、移動して初めて表面化する（表10）。同じ言語を名乗る二つの処理系が同じソースを同じ意味で受け入れることは、検査しない限り成り立たない。第二に、実機の主の時間制約が、エージェントの書き方を状態機械へ強いた。この制約は欠点だけではない。各断片の境界ではエージェントの状態がすべてフィールドにあるため、断片の間で状態を保存して運ぶ、より強い移動性への足場になる。また、LLMが直接書く作業プログラムは`now r.pick(id)`のように応答を待つ形であり、本稿の状態機械とは形が異なる。シミュレータの主のコンパイラは、メソッドを`now`の位置で継続に分割する処理をすでに持っており、これを使って待つ形のプログラムを断片の列へ変換できる見込みがある。この変換が両方の主で同じ意味を保つかは未検証である。

主な制約は知覚の誤り、摩擦と把持力学の近似、カメラと機体の較正差、LLMのプロンプトや利用可能なActor操作への依存である。小規模な課題集合から自然言語一般への適用範囲を推定することもできない。シミュレーションと実機が異なる場合、作業論理、場面推定、運動計画、接触モデル、制御器のどこで差が

生じたかを分析する。既存のプログラム合成・修正研究を踏まえ、概念上の新規性を過大に主張せず、異なる実装と測定された効果を示す必要がある[6, 7]。実時間制御と複数ロボットへの拡張は別の研究課題である。

9　おわりに

本稿は、自然言語から生成したAIPLの作業プログラムをモバイルエージェントとして扱い、シミュレータの主で実行して合格したものだけを、同じソースのまま実機の主へ移動させる構成を定めた。手で書いた振り分けのエージェントは、シミュレータと実機の両方で2個の立方体を運び、同じソースが二つの主で動くことを示した。その過程で、同じソースを受け入れるために揃えるべき処理系の差を列挙した。事前検査の段階と、シミュレーション選別の効果を測る対応のある実験計画も示した。実証的な貢献は今後の実験に依存する。最終稿の結論は、測定された効果とその成立条件に限って記述する。

投稿前に追記・確認する事項

シミュレータの仕様を固定する（Actor契約の実装と擬似コードの差し替えは完了した）。港にシミュレーションの合否による移動の関門（式(1)）を実装する。Actor契約の各操作を両方の主が与える操作として実装し、生成したエージェントを移動させる。実機のXinuの三つの差（フィールドの初期値、コメント中の非ASCII文字、`web_expose`の結び付け）をカーネル側で直し、カメラが止まる原因を調べる。機体とソフトウェアの版を特定し、計画した実験と結果を追加する。必要に応じてシステム構成図と実験写真を掲載し、要旨、考察、結論を結果に合わせて改稿する。投稿先が決まり次第、その書式と開示規則に従う。

参考文献

[1] M. Ahn et al., “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” 2022. https://arxiv.org/abs/2204.01691.

[2] J. Liang et al., “Code as Policies: Language Model Programs for Embodied Control,” 2022. https://arxiv.org/abs/2209.07753.

[3] W. Huang et al., “VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models,” *Conference on Robot Learning*, 2023. https://proceedings.mlr.press/v229/huang23b.html.

[4] P. Hua et al., “GenSim2: Scaling Robot Data Generation with Multi-modal and Reasoning LLMs,” *Conference on Robot Learning*, PMLR 270, pp. 5030–5066, 2025. https://proceedings.mlr.press/v270/hua25a.html.

[5] X. Li et al., “Evaluating Real-World Robot Manipulation Policies in Simulation,” *Conference on Robot Learning*, PMLR 270, pp. 3705–3728, 2025. https://proceedings.mlr.press/v270/li25c.html.

[6] J. Holtz, A. Guha, and J. Biswas, “Robot Action Selection Learning via Layered Dimension Informed Program Synthesis,” *Conference on Robot Learning*, PMLR 155, pp. 1471–1480, 2021. https://proceedings.mlr.press/v155/holtz21a.html.

[7] W. Wang, Y. Rong, Y. Li, L. Jiao, and J. Yuan, “LLM-Driven Corrective Robot Operation Code Generation with Static Text-Based Simulation,” arXiv:2512.02002, version 3, 2026. https://arxiv.org/abs/2512.02002.

[8] A. Fuggetta, G. P. Picco, and G. Vigna, “Understanding Code Mobility,” *IEEE Transactions on Software Engineering*, vol. 24, no. 5, pp. 342–361, 1998.

[9] D. B. Lange and M. Oshima, *Programming and Deploying Java Mobile Agents with Aglets*, Addison-Wesley, 1998.

[10] D. B. Lange and M. Oshima, "Seven Good Reasons for Mobile Agents," *Communications of the ACM*, vol. 42, no. 3, pp. 88–89, 1999.

[11] G. Agha, *Actors: A Model of Concurrent Computation in Distributed Systems*, MIT Press, 1986.